DuQuantum Participant Pre-Reading

Pasqal Tutorials, Tools & Papers

Green underlined text indicates a clickable link to an external resource.

Get ready for Pasqal Challenge

Learn the workflow **optimization model → atom register → laser pulses → measured bitstrings**, then run a small example. If optimization is new to you, start with the IBM resources below, then follow the three core Pasqal tutorials in §2. Use the other resources as needed; you do not need to read every paper in full.

Audience: Participants with computer science, mathematics, operations research, or physics backgrounds.
Context: Analog quantum optimization with neutral atoms. The challenge problem will be revealed at the hackathon.
Scope: A preparation guide—not the final challenge rules or a model solution. Study times are suggested budgets.

Official starting points: Pasqal · Documentation hub · Programming ecosystem · Software · Cloud · Community overview · Community forum · GitHub organization. Some community content may require an account.

Optimization basics — recommended first

• **IBM Decision Optimization tutorials** — introductory optimization notebooks.

• **Start with the Linear Programming notebook** to learn the basic concepts: decision variables, objective functions, constraints, feasible solutions, and optimal solutions.

Focus on how a real-world problem becomes a mathematical model. Linear programming introduces optimization fundamentals; QUBO modeling is a separate step covered in the Pasqal resources below.

1. Install first: set up your environment

Set up before the event with **Python 3.10+**. Start with Pulser; add other libraries only as needed.

Official installation guides:

• Pulser — core library

• QoolQit — dimensionless program design

• Qubo Solver — installation and first solve

• EMU-SV / EMU-MPS — local emulators

• Pasqal Cloud SDK — remote execution

Ex)

1. Create and activate a virtual environment.

2. Install the core library (Pulser). This also installs `pulser_simulation` for local emulation.

3. Add the other libraries you plan to use.

4. Check that the install worked by running the first tutorial in §1.

```
# 1. Virtual environment
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate

# 2. Core library (required)
pip install pulser

# 3. Optional libraries (install what you need)
pip install qoolqit              # dimensionless program design
pip install qubo-solver          # end-to-end QUBO baseline
pip install emu-sv emu-mps       # Pasqal emulators (state vector / MPS)
pip install pasqal-cloud         # cloud emulator and QPU jobs
```

Use the organizers' pinned environment if provided. Cloud/QPU access requires organizer credentials; never include them in notebooks, repositories, or slides.

2. Start here: core tutorials and a QUBO demo

1. **Programming with Pulser** · 30–45 minutes

• Follow single-atom excitation and multi-atom adiabatic state preparation. Learn how registers, sequences, pulses, Ω, and δ connect to measured outcomes.

2. **Register and Rydberg-Atom Interactions** · 20–30 minutes

• Learn how distances set interactions and why an arbitrary QUBO cannot necessarily be implemented directly on an atom array.

3. **Qubo Solver** · 30–45 minutes (estimated)

• Learn to solve a QUBO; complete the installation and first solve in §1 first.

4. **QUBO demo notebook — unitaryCON Pasqal Demo** — additional hands-on example.

Videos

• Rydberg Quantum Processor Basics

• QoolQit: the new Pasqal library for analog quantum programming · from 19:06 — introduction to designing and compiling analog quantum programs with QoolQit

• Webinar: Quantum Computing as a Service — Discover Pasqal's Cloud Platform — overview of the Pasqal cloud platform

3. Essential vocabulary

Concept	What it means here
Hamiltonian	The operator governing energy and time evolution. The goal is to connect good solutions to low-energy configurations.
Rabi frequency Ω	The strength of laser driving between ground and Rydberg states.
Detuning δ	The offset from atomic resonance; it affects the energy cost or reward of excitation. Global detuning is shared by all atoms; local detuning varies by atom.
Rydberg interaction	In the repulsive model used here, two excited atoms incur a positive interaction proportional to $1/r^6$.
Adiabatic evolution	Changing the Hamiltonian slowly relative to the energy gap to follow its ground state under suitable conditions. Finite duration and noise prevent a general guarantee of optimality.
QUBO / slack variable	A quadratic cost over binary variables; constraints can be encoded with penalties. Slack variables convert inequalities to equalities and may require extra bits—and atoms.
Register / embedding	Atom positions / mapping the target model to physically realizable interactions and controls.
Shot / bitstring	One measurement / its 0–1 output, which must be decoded and checked against the original constraints.

4. Tools: tutorials, documentation, and GitHub

Start with Pulser and a small local example. Add other tools only when needed.

Tool	Learning resources	Source code
Pulser	Core tutorials in §2; pulse optimization for iterative parameter tuning	Pulser
QoolQit	Programming with QoolQit: dimensionless program design and device compilation	qoolqit
Qubo Solver	Documentation: baselines, embedding, drives, and result analysis. Installation in §1; tutorial in §2.	qubo-solver
EMU-SV / EMU-MPS	Emulators Cloud Integration: backend and cloud execution guidance	emulators
Pasqal Cloud SDK	Installation in §1; cloud integration guidance in the emulator row above. Follow organizer access instructions.	Repository linked in §1

5. Recommended papers

Platform background — priority reading

Henriet et al. (2020), “Quantum computing with neutral atoms”
Free arXiv version · Published version

Read for optical tweezers, Rydberg interactions, and the distinction between analog and digital operation. Start with the abstract, figures, and platform overview—not historical hardware limits.

Pulse-level software — priority reading

“Pulser: An open-source package for the design of pulse sequences in programmable neutral-atom arrays” (2022)
Free arXiv version · Published version

Read for the register–channel–pulse–sequence abstractions and the role of device constraints and emulation. Use current documentation for API details.

QUBO modeling

Glover, Kochenberger & Du, “A Tutorial on Formulating and Using QUBO Models”
Free arXiv version; first posted in 2018

Focus on binary variables, constraint penalties, and checking small formulations. A mathematically correct QUBO is not necessarily directly embeddable on neutral-atom hardware.